



المدرسة العليا للمواصلات بتونس
Ecole Supérieure des Communications de Tunis
Higher School of Communications of Tunis

TECHNICAL REPORT – P2M PROJECT

Design and Development of an AI-Based Network Anomaly Detection System Using NetFlow

Done By :
Ahmed Abdelhedi
Morsi Feki

Supervised By:
Prof. Slim Rekhis

Academic Year: 2025/2026

Table of Contents

1. Introduction	1
1.1 Context and Motivation (what is NetFlow?)	1
1.2 Problem Statement.....	1
1.3 Objectives.....	2
2. Proposed Solution Architecture	2
2.1 Overview of the System.....	3
2.2 Network Infrastructure (VirtualBox Environment).....	3
2.3 NetFlow Version.....	4
2.4 NetFlow Collection and Processing Pipeline.....	4
2.5 SIEM Integration with Wazuh.....	4
2.6 Manual Detection using Wazuh Dashboards.....	5
a) SYN Flood DoS Attack	
b) SSH Brute Force Attack	
c) DNS Exfiltration Attack	
3. Dataset and Preprocessing	7
3.1 CSE-CIC-IDS2018 Dataset.....	7
3.2 Feature Selection and Encoding.....	7
3.3 Temporal Windowing Strategy.....	7
3.4 Train and test split.....	8
3.5 Class Balancing and Scaling.....	9
4. Transformer-Based Detection Model	9
4.1 Model Architecture.....	9
4.2 Training Configuration and Optimization.....	10
4.3 Model Performance.....	10
5. Deployment and Results.....	11
5.1 Model Deployment in SIEM.....	11
5.2 Experimental Validation.....	12
6. Conclusion	13

1. Introduction

1.1 Context and Motivation

The rapid evolution of cyber threats poses a serious challenge to modern network security. Traditional Intrusion Detection Systems (IDS) rely on signature-based methods that match network packets against known attack patterns. While effective against catalogued threats, these approaches fail to detect highly sophisticated or novel attacks — especially those carried out over encrypted channels where payload inspection is no longer possible.

Modern attacks such as Advanced Persistent Threats (APTs), slow-and-low data exfiltration, command-and-control (C2) communications, and encrypted malware communications leave minimal payload-level evidence but still manifest as observable deviations in network traffic behavior. Detecting these anomalies requires a shift from content-based inspection to flow-level behavioral analysis using for example NetFlow.

What is NetFlow?

NetFlow is a log format for network traffic, originally introduced by Cisco. Instead of capturing the full content of each packet like a PCAP file does, NetFlow describes each network communication at a high level - summarizing what happened between two endpoints without recording what was said. Each NetFlow record describes a single network flow: a conversation identified by its source IP, destination IP, source port, destination port, and transport protocol.

A NetFlow record typically contains the following fields:

- Source and destination IP addresses and port numbers
- Transport protocol (TCP, UDP, ICMP)
- Flow start and end timestamps, giving the total duration of the communication
- Total number of packets and bytes transferred
- TCP flag counts (SYN, ACK, FIN, RST, PSH, URG) observed during the flow

Because NetFlow only records metadata about communications, it works regardless of whether the traffic is encrypted enabling efficient behavioral analysis and supports machine learning-based detection techniques. This makes it particularly well-suited for security monitoring in modern networks, where TLS encryption makes deep packet inspection impractical.

In addition, NetFlow is highly scalable, making it suitable for deployment in large-scale networks.

A NetFlow log cannot tell you what data was exchanged, but it can tell you how much, for how long, and with what behavioral characteristics -which is often sufficient to detect anomalies.

1.2 Problem Statement

Despite the availability of NetFlow data in most enterprise networks, its exploitation for real-time intrusion detection remains limited. The primary challenges are:

- The high dimensionality and volume of NetFlow data make manual analysis impractical at scale.

- Traditional rule-based SIEM systems cannot capture complex temporal dependencies in traffic sequences.
- Sophisticated behavioral anomalies require models capable of generalizing beyond known signatures.
- There is a need for automated, intelligent classification systems that can identify attack patterns from raw NetFlow features and integrate with existing SIEM platforms.

1.3 Objectives

The main objectives of this project are:

1. Implement a complete network monitoring laboratory using VirtualBox with dedicated attacker and victim segments.
2. Collect NetFlow traffic data using tcpdump on pfSense and process it through CICFlowMeter for feature extraction.
3. Integrate the detection pipeline with Wazuh SIEM for real-time alerting and dashboard visualization.
4. Design and train a Transformer-based deep learning model on the CSE-CIC-IDS2018 dataset for automated multi-class attack classification. The dataset provides NetFlow-like bidirectional flow records extracted from raw network traffic.
5. Validate the model's effectiveness across multiple attack categories.

2. Proposed Solution Architecture

2.1 Overview of the System

The proposed system is a multi-layered anomaly detection framework that combines a monitored network laboratory, a SIEM platform, and an AI-based classification engine. The overall data flow follows this pipeline:

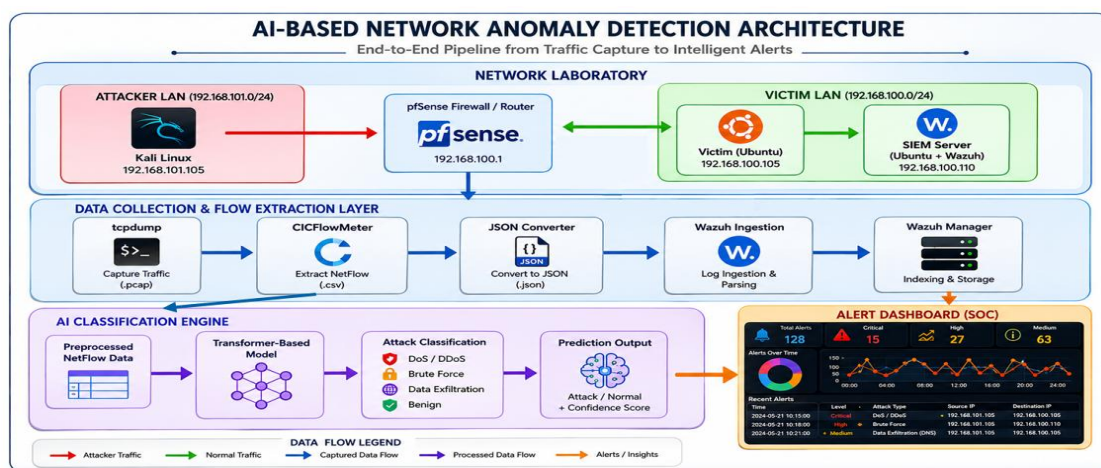


Figure 1: System architecture

This architecture enables end-to-end intrusion detection: from raw packet capture at the network boundary to actionable security alerts presented to the SOC engineer.

2.2 Network Infrastructure (VirtualBox Environment)

The experimental environment was implemented using **Oracle VirtualBox**, providing a controlled and reproducible virtual laboratory for network monitoring and attack simulation. The infrastructure is composed of four virtual machines interconnected through a combination of **Network Address Translation (NAT)** and **two isolated internal networks**, enabling realistic traffic flow and segmentation.

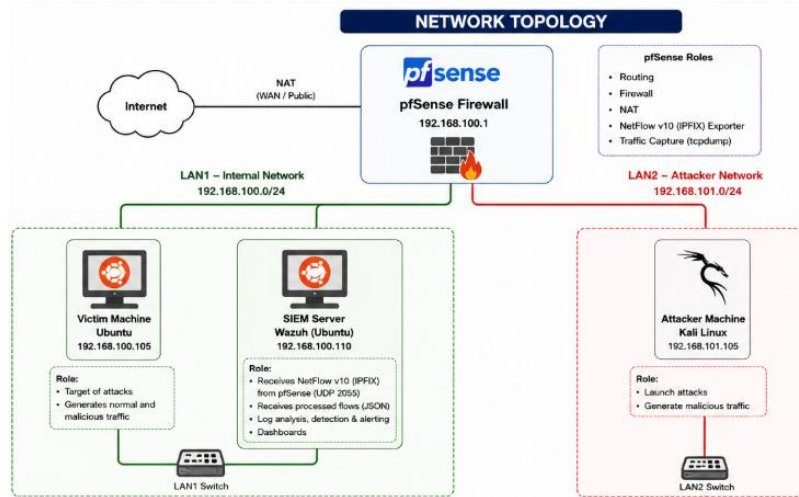


Figure 2: Network Topology

Infrastructure Components:

- **Hypervisor – Oracle VirtualBox**

The virtualization platform used to host and manage all virtual machines. It is configured with one NAT network for internet access and two internal networks to simulate isolated LAN environments.

- **pfSense Gateway (192.168.10.1):** Firewall/router connecting both LANs and capturing traffic using tcpdump.

The pfSense firewall was configured to capture network traffic at the gateway level using tcpdump. This setup ensures that all traffic between network segments is monitored. Although pfSense natively supports NetFlow export (via softflowd), in this project packet capture was preferred to allow detailed post-processing with CICFlowMeter, which generates enriched bidirectional flow records suitable for machine learning.

- **SIEM Server – Ubuntu (192.168.10.50):** Runs Wazuh for log processing, detection, and dashboards.
- **Victim Machine – Ubuntu (192.168.10.101):** Target system hosting services and generating normal/attack traffic.
- **Attacker Machine – Kali Linux (192.168.50.100):** Used to simulate cyberattacks from an isolated network.

2.3 NetFlow Version

NetFlow exists in multiple versions, each introducing improvements in flexibility and extensibility.

NetFlow v5 is one of the most widely used versions, providing a fixed format with basic flow information.

NetFlow v9 introduces a template-based structure, allowing for customizable flow records and greater extensibility.

IPFIX (NetFlow v10) further standardizes this approach and is widely adopted in modern systems.

In this project, NetFlow v10 (IPFIX) was used at the pfSense gateway to generate flow-level traffic summaries directly from the network.

However, to enable more advanced analysis and machine learning capabilities, raw traffic was also captured using **tcpdump** and processed with **CICFlowMeter**. This tool generates enriched bidirectional flow records with over 80 features, providing deeper statistical and temporal characteristics compared to standard NetFlow exports.

Therefore, the system combines native NetFlow v10 monitoring for real-time visibility with **CICFlowMeter**-based flow generation for detailed feature extraction and AI-based anomaly detection.

2.4 NetFlow Collection and Processing Pipeline

Network traffic is monitored at the pfSense gateway using two complementary approaches. First, **NetFlow v10 (IPFIX)** is enabled on pfSense to generate real-time flow-level summaries of network communications.

In parallel, raw network traffic is captured using **tcpdump** at the firewall, generating **PCAP files**. These files are transferred to the SIEM server and processed with **CICFlowMeter** to extract enriched bidirectional flow features.

CICFlowMeter produces more than **80 features** per flow, including flow duration, packet and byte counts, TCP flag statistics, inter-arrival times, packet length metrics, and throughput indicators such as Flow Bytes/s and Flow Packets/s. These features provide deeper statistical and temporal insights compared to standard NetFlow records, making them well-suited for machine learning-based analysis.

The resulting flow data is exported in **CSV format** and then converted into **JSON** using a script to ensure compatibility with Wazuh for log ingestion, indexing, and further analysis.

2.5 SIEM Integration with Wazuh

Wazuh is an open-source Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) platform used for threat detection, log analysis, and security monitoring.

Why Wazuh?

We selected Wazuh due to its open-source nature, its scalability, its strong support for log analysis, its real time alerting and integration with custom data sources such as NetFlow-derived records.

In this architecture, Wazuh is configured with custom local rules to parse JSON-formatted NetFlow records generated from the previous processing pipeline. Each record is indexed and made searchable within the Wazuh dashboard.

We used Wazuh to:

- Monitor real-time flow statistics and TCP flag distributions
- Build custom dashboards using time-series visualizations and top-N traffic analysis
- Identify anomalous traffic patterns that may indicate known attack behaviors
- Receive automated alerts generated by the AI-based classification module

2.6 Manual Detection using Wazuh Dashboards

To validate the proposed intrusion detection framework, attacks were launched in the experimental environment to evaluate the **detection capabilities of the SIEM dashboards**. The validation was performed on the deployed laboratory infrastructure composed of the attacker machine (**Kali Linux**), the victim machine (**Ubuntu**), the **pfSense** gateway, and the **Wazuh SIEM server**.

Three attack scenarios were considered: **SYN Flood DoS**, **SSH Brute Force**, and **DNS Exfiltration** attacks.

a) SYN Flood DoS Attack

A **Denial-of-Service (DoS)** attack based on **TCP SYN flooding** was generated from the attacker machine. The objective of this attack is to create a large number of incomplete TCP connections by repeatedly sending SYN packets without completing the TCP handshake.

The attack produced abnormal NetFlow characteristics, including:

- High packet rate (**Flow Packets/s**)
- Large number of flows containing only the **SYN flag** ()
- Very short **Flow Duration**
- Numerous incomplete TCP sessions

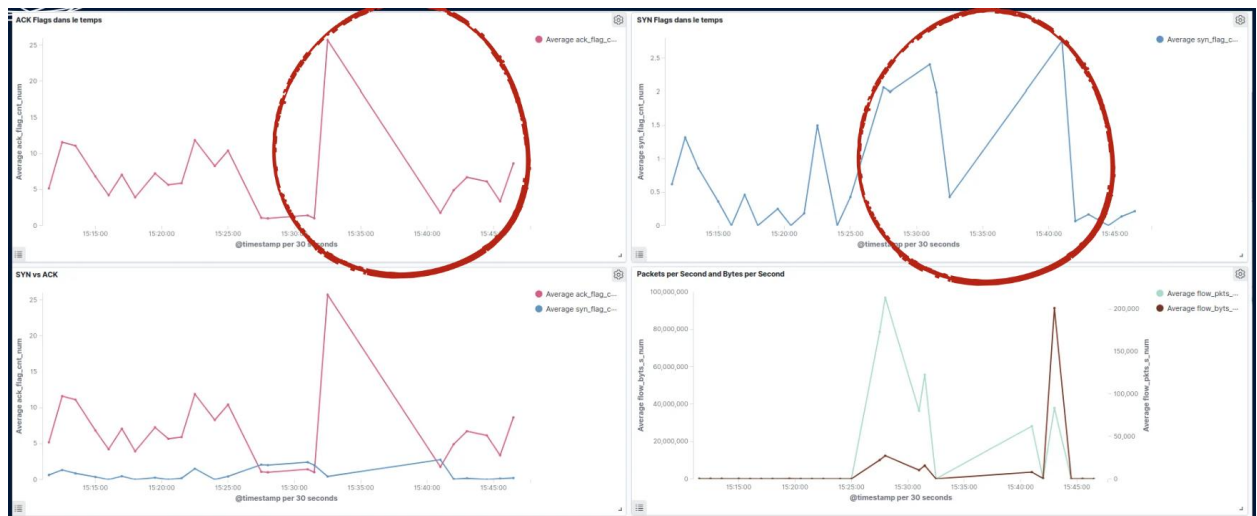


Figure 3: Temporal Analysis of SYN and ACK Flags

b) SSH Brute Force Attack

An **SSH brute force attack** was launched against the victim machine by generating repeated authentication attempts on **port 22**.

The attack generated a large number of short-lived connections with repetitive communication patterns. The following NetFlow indicators were observed:

- Repeated connections targeting **Destination Port = 22**
- Short session durations
- Low response traffic from the victim machine
- Multiple flows originating from the same source address

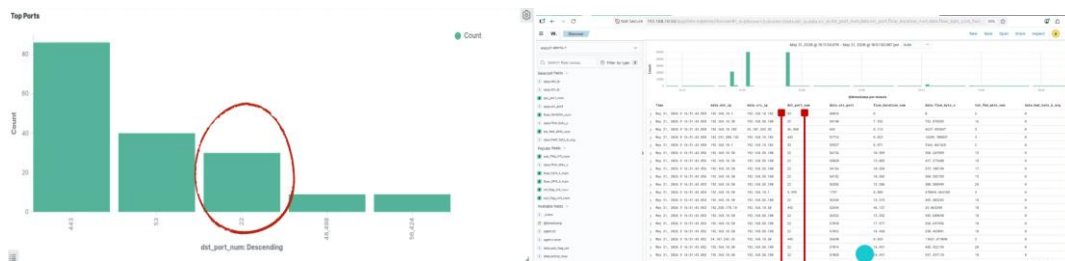


Figure 4: Detection of SSH Brute Force Activity Using NetFlow Features

c) DNS Exfiltration Attack

A **DNS-based exfiltration scenario** was also evaluated to simulate covert data transfer through DNS traffic.

This attack generated unusual DNS communication patterns characterized by:

- High frequency of DNS queries (**Port 53**)

- Larger-than-normal DNS packet sizes
- Abnormal payload volumes
- Irregular communication behavior over time



Figure 5: Packets per Second and Bytes per Second

The generated NetFlow traces exhibited suspicious variations in features such as **Packet Length Mean**, **Flow Bytes/s**, and **Subflow Forward Bytes**.

The experimental results demonstrate that the proposed solution is capable of detecting multiple attack categories using only **NetFlow information**, without requiring packet payload inspection. However, **manual detection remains challenging**, as SOC analysts must continuously monitor dashboards and network indicators in real time to identify suspicious behaviors. This process is time-consuming, requires constant supervision, and may lead to missed attacks, especially for subtle or slow-evolving threats. Therefore, integrating the AI-based detection model enables the automation of the detection process and reduces the burden on analysts.

3. Dataset and Preprocessing

3.1 CSE-CIC-IDS2018 Dataset

The CSE-CIC-IDS2018 dataset, published by the Canadian Institute for Cybersecurity, is a widely used benchmark for network intrusion detection research. It contains realistic network traffic collected over several days in a controlled environment, with both benign and attack traffic generated by dedicated machines.

Reference: <https://www.unb.ca/cic/datasets/ids-2018.html>

The CSE-CIC-IDS2018 dataset was selected due to its realism, diversity of attack scenarios, and availability of flow-based features compatible with NetFlow analysis. It provides labeled traffic generated in a controlled environment, making it suitable for supervised learning and benchmarking intrusion detection models.

The dataset includes traffic for the following attack categories relevant to this project:

- DoS attacks — including SYN flooding, HTTP flooding, and Slowloris

- Brute Force attacks — targeting SSH and FTP authentication services
- Infiltration and exfiltration attacks — including DNS tunneling and data theft scenarios
- Benign traffic — covering standard enterprise network activity

3.2 Feature Selection and Encoding

The raw dataset contains over 80 features per flow record. A feature selection step was applied to remove:

- Features that may cause overfitting (ip addresses, Flow id, ports ...)
- Constant or near-zero variance features
- Redundant features with pairwise correlation > 0.95

The resulting feature set retained the most discriminative flow-level attributes. The label column, containing attack class names as strings, was processed using One-Hot Encoding to produce a multi-class binary matrix. Categorical string features were similarly encoded.

3.3 Temporal Windowing Strategy

The Transformer model operates on sequences of flow records rather than individual records, enabling it to capture temporal dependencies between successive flows. A sliding window approach was adopted: consecutive rows ordered by timestamp are grouped into fixed-size windows.

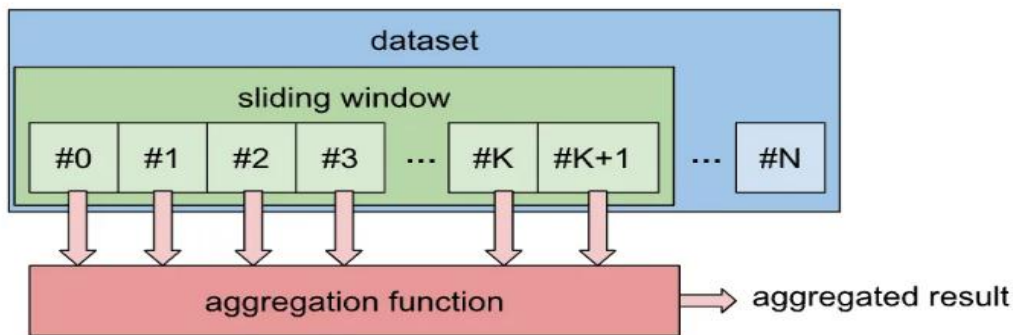


Figure 6: Sliding Window-Based Data Aggregation Process

Window = 10 rows: Threshold = 0.1 — A window is labelled as an attack if more than 10% of its rows belong to an attack class

Window = 50 rows: Threshold = 0.02 — A window is labelled as an attack if more than 2% of its rows belong to an attack class

The window of 50 rows with a threshold of 0.02 was ultimately selected as it provides a longer temporal context, allowing the model to detect slow or distributed attack patterns that unfold gradually over time.

3.4 Train and test split

After windowing, the dataset was split into training and test sets using an 80/20 stratified split to preserve class proportions.

3.5 Class Balancing and Scaling

The training set required further balancing:

- Attack classes with fewer than a minimum window count were removed to avoid training on statistically insufficient samples.
- The dominant Benign class was undersampled to prevent the model from developing a bias toward predicting normal traffic.

Finally, all feature values were standardized using a **StandardScaler** fitted exclusively on the training data and applied to both training and test sets, preventing any data leakage.

4. Transformer-Based Detection Model

The proposed intrusion detection model is built upon a **Transformer Encoder architecture**, selected for its strong ability to capture long-range dependencies within sequential data using self-attention mechanisms. Unlike recurrent neural networks (RNNs), Transformers process the entire input sequence in parallel, which enables more efficient training and improved modeling of complex temporal relationships present in NetFlow time series.

4.1 Model Architecture

The NetflowTransformer model is structured as follows:

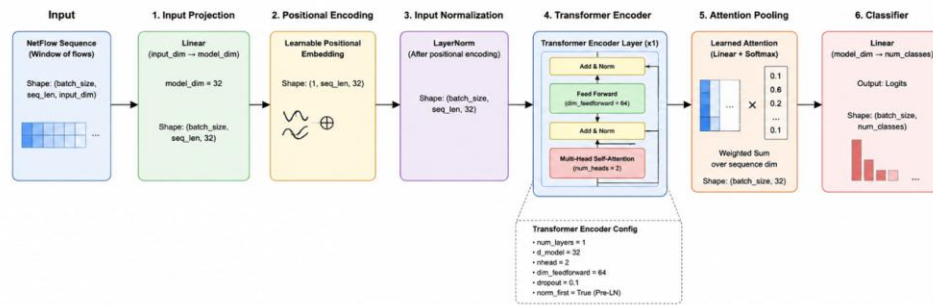


Figure 7: Transformer Model Architecture

The proposed intrusion detection model is based on a **Transformer Encoder architecture** designed to analyze temporal dependencies in NetFlow sequences. As shown in this **Figure**, the model first projects raw NetFlow features into an embedding space of dimension **32** using an input projection layer. A **learnable positional encoding** is then added to preserve temporal information within the sequence.

The encoded inputs are normalized using **LayerNorm** and processed by a **Transformer Encoder** composed of **1 layer**, **2 attention heads**, a **feed-forward dimension of 64**, and **dropout = 0.1**. To aggregate sequence information, an **attention pooling mechanism** is applied to emphasize the most informative time steps, particularly useful for detecting short bursts of anomalous activity. Finally, a linear classifier outputs the predicted attack class.

4.2 Training Configuration and Optimization

The model was trained using the following hyperparameter configuration:

Parameter	Configuration
Loss Function	CrossEntropyLoss — standard objective for multi-class classification
Optimizer	AdamW with learning rate = 3×10^{-4} and weight decay = 1×10^{-4}
Scheduler	CosineAnnealingLR with $T_max = 10$ and $\eta_min = 1 \times 10^{-6}$
Batch Size	128 windows per batch, with <i>drop_last = True</i> to avoid biased final batches
Epochs	Maximum 30 epochs with early stopping
Gradient Clipping	Maximum norm = 1.0 to prevent gradient explosion

Table 1 : Hyperparameter Configuration for Transformer Model Training

The selected hyperparameters were determined empirically through multiple experimental trials. Different configurations were tested, and the final setup was chosen based on validation performance and training stability rather than exhaustive hyperparameter optimization.

4.3 Model Performance

The model achieved high classification performance, with accuracy and macro F1-score exceeding 99%. This demonstrates the effectiveness of Transformer architectures in capturing temporal dependencies in NetFlow sequences.

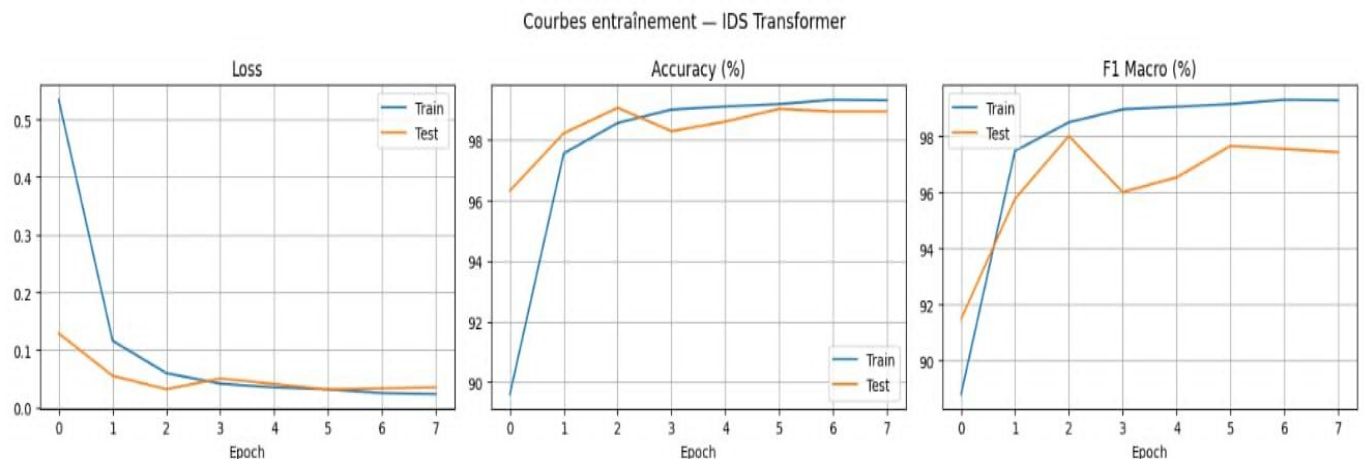


Figure 8 : Training and Evaluation Curves of the Transformer-Based IDS Model: Loss, Accuracy, and Macro F1-Score



Figure 9 : Confusion Matrix

5. Deployment and Results

5.1 Model Deployment in SIEM

After training and validating the Transformer model, it was integrated into the SIEM pipeline to enable automatic intrusion detection from NetFlow traffic. The process begins with traffic collection and ends with alert visualization in the SOC dashboard.

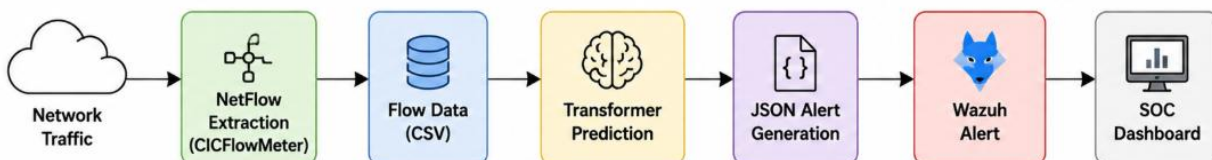


Figure 10: SIEM Integration Pipeline

The model is deployed as a local script on the SIEM machine. NetFlow records are preprocessed and passed to the model for inference, and the predicted labels are then integrated into Wazuh for alert generation and dashboard visualization.

5.2 Experimental Validation

A Slow HTTP attack was conducted using the **SlowHTTPTest** tool from the attacker machine. The attack generated a large number of HTTP connections while sending HTTP headers at a very slow rate, keeping the connections open for an extended period. This behavior aimed to exhaust the victim server's available connection resources and degrade its ability to serve legitimate clients.

The attack resulted in abnormal traffic patterns, including a high number of long-lived HTTP connections, incomplete HTTP requests, and an unusual increase in concurrent sessions. These indicators were successfully detected by the monitoring system and generated security alerts in Wazuh.

Example of Slow HTTP attack detection:

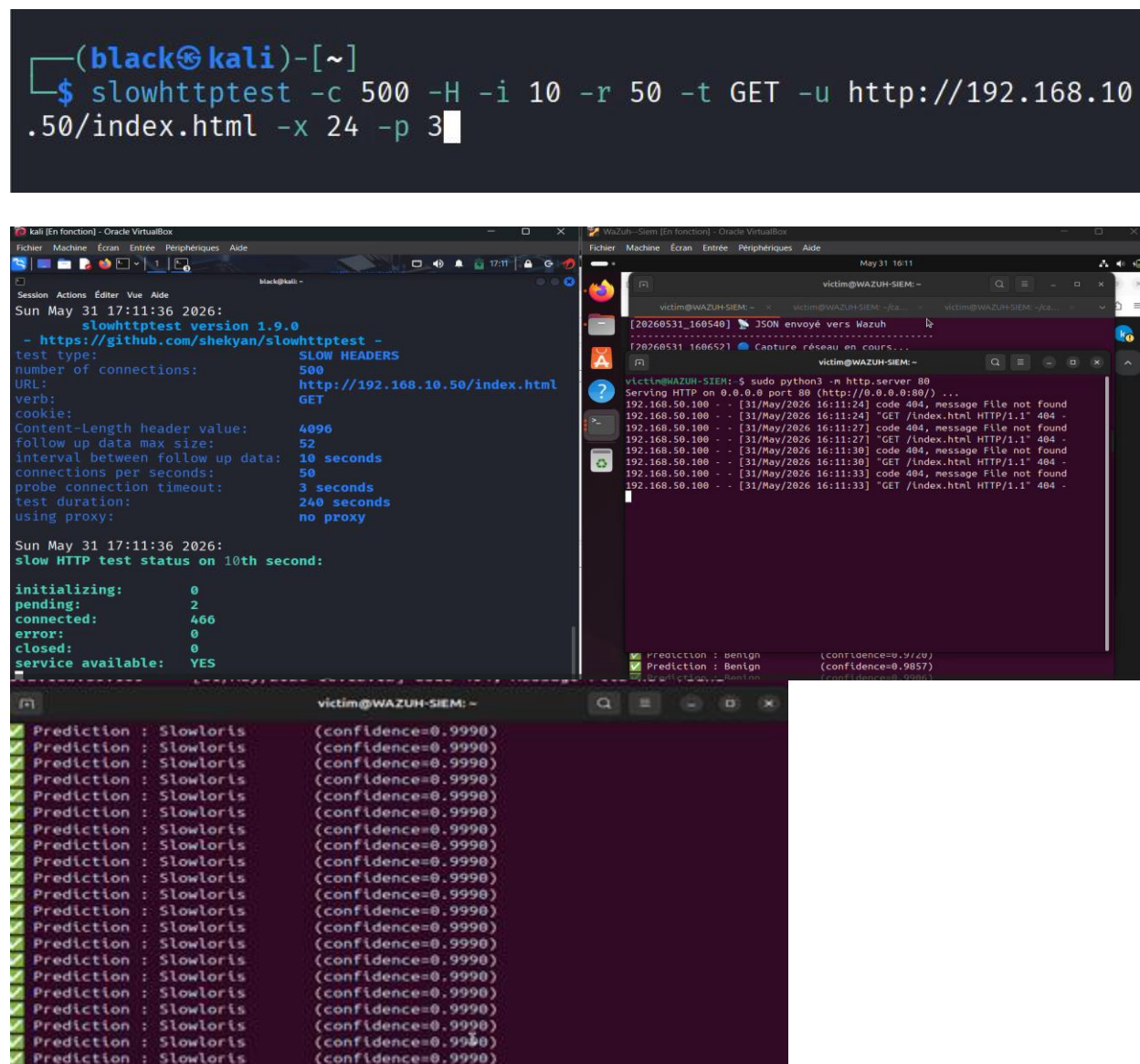


Figure 11: Slow HTTP Attack Scenario

Alerts printed in the Wazuh interface:

Date	Time	Source IP	Destination IP	Protocol	Score
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.310895
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.327585
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.330997
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.330928
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.341889
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.340434
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.357160
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.345758
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.346584
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.343337
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.435471
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.371890
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.357382
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.377441
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.389346
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.382039
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.367566
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.365470
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.398476
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.390582
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.380389
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.437383
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.379854
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.402560
May 25, 2020	14:52:42	192.168.0.100	192.168.10.10	TCP	0.381336

Figure 12: Visualization of Detected Intrusions in the Wazuh Dashboard

6. Conclusion

This project demonstrated the feasibility and effectiveness of an AI-based network anomaly detection system built on top of standard SIEM infrastructure. By combining NetFlow data collection at the network boundary, feature extraction via CICFlowMeter, SIEM ingestion through Wazuh, and a Transformer-based deep learning classifier, the system achieves a test accuracy and macro F1-score exceeding 99% across three distinct attack categories: DoS SYN Flooding, SSH Brute Force, and DNS Exfiltration.

The Transformer architecture proved particularly well-suited to this problem, as its self-attention mechanism enables the model to identify subtle temporal dependencies in flow sequences — a capability that is essential for detecting slow, low-volume, or covert attack behaviors that evade signature-based systems.

The integration with Wazuh provides a practical deployment path: classification results are surfaced alongside traditional SIEM alerts, enabling SOC engineers to benefit from AI-assisted detection without replacing their existing workflows.

Several directions are planned to extend and improve the system:

- **Unsupervised Autoencoder:** Training a complementary autoencoder on benign traffic to detect zero-day anomalies through reconstruction error, enabling the identification of threats not represented in the training dataset.
- **Model Generalization:** Improving the Transformer model generalization by training it on multiple intrusion detection datasets instead of a single dataset, allowing better adaptation to diverse network environments and attack scenarios.
- **XDR Integration with Wazuh:** Extending the current SIEM deployment by integrating Extended Detection and Response (XDR) capabilities within Wazuh to enable automated response and mitigation actions.
- **Adversarial Robustness:** Evaluating model performance against adversarially crafted NetFlow records designed to evade the classifier and assessing the resilience of the detection system.